PROJECT REPORT

INDICMEASURE

Historical & Traditional Indian Unit Converter

Submitted by:

- **Janamejay Deval** – Student ID: 1262240200
- **Lavkesh Mutha** – Student ID: 1262240309

A Java-based web application using Java, JDBC, SQLite, HTML, CSS and JavaScript

TABLE OF CONTENTS

1. Introduction

2. Objectives

3. Technologies Used

4. System Architecture

5. Project Development

6. Conversion Logic

7. Database and Unit Management

8. User Interface and Features

9. Testing and Results

10. Challenges and Observations

11. Conclusion

12. Images and Demonstration

Annexure – Project Structure

1. Introduction

IndicMeasure is a web-based unit conversion project designed to connect historical and traditional Indian measurements with the modern metric system. The application allows users to select a measurement system, category and unit, enter a value, and obtain a converted result.

The project supports traditional units such as Tola, Masha, Ratti, Pau, Seer, Maund, Gaz, Gaj, Kos and Yojana, together with modern metric units. Historical measurements can vary by period and region, so the factors used by the application are treated as standardized or approximate values.

The application was developed using Java for the backend, SQLite for persistent unit data, JDBC for database connectivity, and HTML, CSS and JavaScript for the user interface.

2. Objectives

- Develop a simple and user-friendly unit conversion application.
- Provide conversions between traditional Indian and metric units.
- Support Mass, Length, Volume and Area categories.
- Implement both forward and reverse conversion.
- Store unit information and conversion factors in SQLite.
- Apply Java object-oriented programming concepts.
- Create a professional web interface.
- Maintain conversion history and display unit information.

3. Technologies Used

Technology	Purpose	Application in Project
Java	Backend programming	Conversion logic, HTTP server and application structure
SQLite	Database	Stores units, categories, factors and descriptions
JDBC	Database connectivity	Connects Java application to SQLite
HTML	Frontend structure	Converter interface and information panels
CSS	Frontend styling	Professional and responsive interface
JavaScript	Frontend logic	API communication, filtering and history
VS Code	Development environment	Project development and testing

4. System Architecture

IndicMeasure follows a client-server architecture. The frontend collects user input and communicates with the Java backend through HTTP requests. The backend retrieves unit information from SQLite and performs conversion using a common base-unit approach.

FRONTEND
HTML + CSS + JavaScript
↓

HTTP API

↓

JAVA SERVER

↓

CONVERSION SERVICE

↓

JDBC

↓

SQLITE DATABASE

The backend is divided into model, service, database and server components. Unit classes represent measurement objects, ConversionService handles conversion operations, DatabaseConnection manages SQLite connectivity, and Server exposes the API endpoints.

5. Project Development

5.1 Database Development

A SQLite database was created to store unit name, category, conversion factor, description, region and system. The system field distinguishes Traditional and Metric units.

5.2 Backend Development

The Java backend was implemented using the built-in HttpServer and JDBC. It provides an API for retrieving units and another API for performing conversions. Validation prevents conversions between incompatible categories.

5.3 Frontend Development

The frontend was developed using HTML, CSS and JavaScript. It contains separate From and To selections, category filtering, value input, a result panel, unit information and conversion history.

5.4 Supported Units

Category	Traditional Units	Metric Units
Mass	Tola, Masha, Ratti, Pau, Seer, Maund	Milligram, Gram, Kilogram, Metric Tonne
Length	Gaz, Gaj, Kos, Yojana	Millimetre, Centimetre, Metre, Kilometre
Volume	—	Millilitre, Litre
Area	—	Square Centimetre, Square Metre, Square Kilometre

6. Conversion Logic

The main conversion principle is conversion through a common base unit. Instead of creating a separate formula for every possible pair, each unit stores a factor relative to its category's base unit.

6.1 Traditional to Metric

Metric Value = Traditional Value × Conversion Factor

10 Tola × 11.6638 = 116.638 Gram

6.2 Metric to Traditional

Traditional Value = Metric Value ÷ Conversion Factor

116.638 Gram ÷ 11.6638 = 10 Tola

6.3 Unit-to-Unit Conversion

For a conversion such as Tola to Masha, the application first converts Tola to the base unit Gram and then converts the Gram value into Masha. This makes the algorithm reusable and easier to extend.

7. Database and Unit Management

SQLite was selected because it is lightweight and does not require a separate database server. The DatabaseManager initializes the database and inserts missing units without creating duplicates when setup is run again.

Field	Purpose
Name	Name of the unit
Category	Mass, Length, Volume or Area
conversion_factor	Factor relative to the category base unit
Description	Basic information about the unit
Region	Historical or regional usage
System	Traditional or Metric

8. User Interface and Features

- From and To system selection.
- Category-based unit filtering.
- Traditional and metric conversion in both directions.
- Clear conversion result display.
- Unit information showing system, category, description and regional usage.
- Conversion history and Clear History function.
- Responsive layout.
- Indian heritage-inspired background with a professional card-based interface.

The interface balances a modern application experience with the historical theme of the project. The heritage background provides visual identity while the converter cards and controls maintain readability.

9. Testing and Results

Representative tests were performed across the supported categories and in both conversion directions.

Test	From	Value	To	Expected Result
1	Tola	10	Gram	116.638 Gram
2	Gram	116.638	Tola	10 Tola
3	Gaz	10	Metre	9.144 Metre
4	Litre	2	Millilitre	2000 Millilitre
5	Square Metre	1	Square Centimetre	10000 Square

				Centimetre

The core conversion tests produced the expected results. The backend successfully loaded 23 units from the database, and the frontend successfully communicated with the Java conversion API.

10. Challenges and Observations

1. Database integration

Connecting SQLite with Java through JDBC required correct classpaths, database initialization and consistent table fields.

2. Multiple measurement systems

The frontend had to support Traditional and Metric units independently while keeping category filtering correct.

3. Reusable conversion logic

Using base-unit conversion avoided creating individual formulas for every pair of units.

4. Frontend-backend consistency

Unit names, categories and system values returned by the API had to match frontend filtering logic.

5. Professional UI design

Several styling iterations were required to achieve a clean interface while preserving the Indian heritage theme.

Observations

- A database-driven design makes adding or updating units easier than hard-coding all conversion values in the frontend.
- A common base-unit approach keeps the conversion algorithm simple and reusable.
- Separating frontend, backend, service and database responsibilities makes the project easier to maintain.
- Reverse conversion testing is useful for detecting incorrect factors and mappings.

11. Conclusion

IndicMeasure successfully demonstrates how a Java-based web application can present traditional Indian measurement systems alongside modern metric measurements. The project combines a SQLite database, JDBC, Java backend services and a responsive HTML/CSS/JavaScript frontend into a single working system.

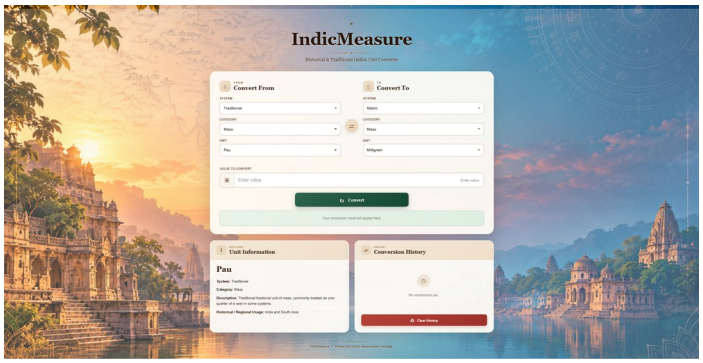
The project provided practical experience in object-oriented programming, database management, API communication, frontend development, validation, testing and user interface design. The common base-unit approach makes the conversion engine scalable, while the database-driven design allows new units to be added more easily.

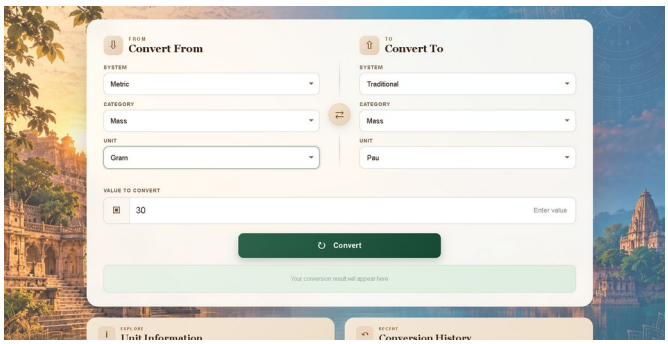
Overall, IndicMeasure provides a functional foundation for a historical measurement reference and conversion platform. It demonstrates both the technical implementation of a multi-category converter and the importance of presenting historical information through a modern, accessible interface.

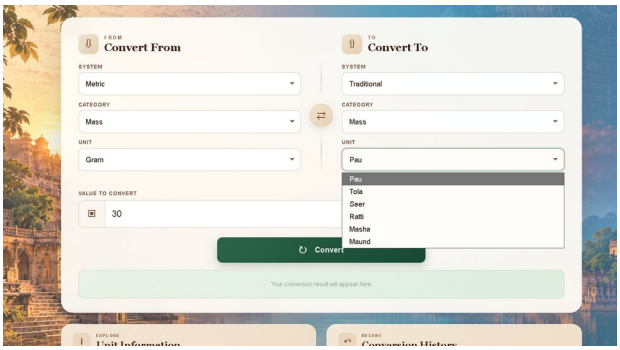
ANNEXURE – PROJECT STRUCTURE

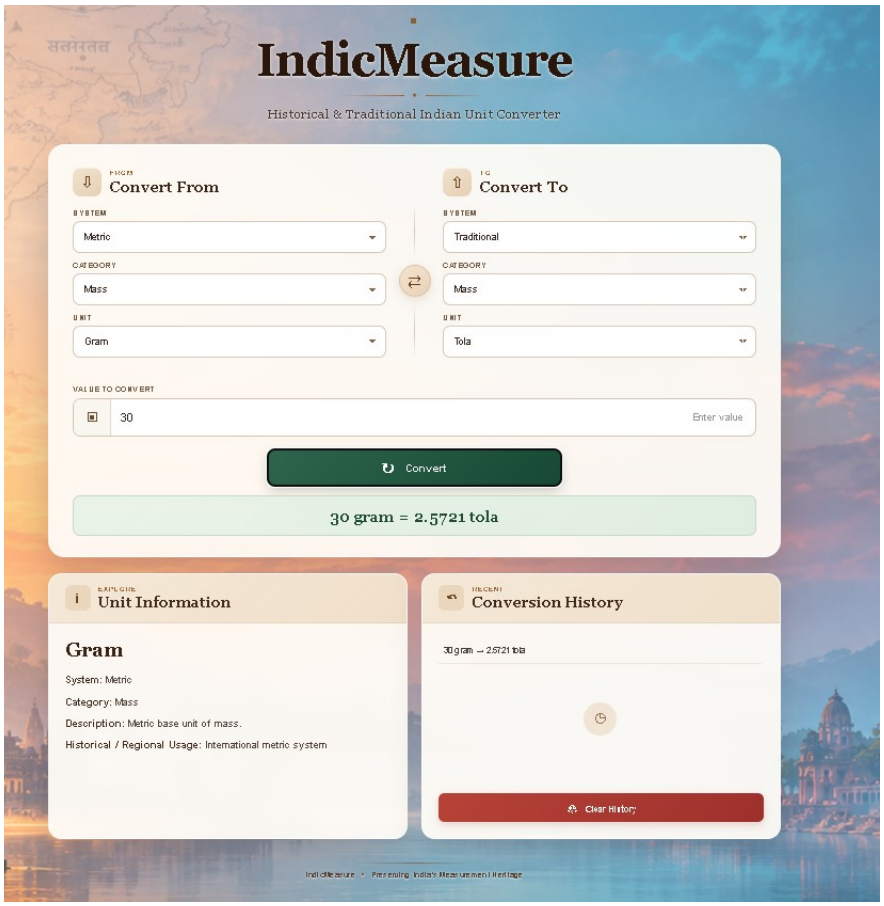
```
Java MicroProject/
├── backend/
│   ├── DatabaseManager.java
│   ├── Server.java
│   ├── database/DatabaseConnection.java
│   ├── model/Unit.java
│   ├── model/MassUnit.java
│   ├── model/LengthUnit.java
│   ├── service/ConversionService.java
│   └── lib/sqlite-jdbc-3.53.4.0.jar
├── database/
│   └── indicmeasure.db
└── frontend/
    ├── index.html
    ├── script.js
    ├── style.css
    └── background.png
```

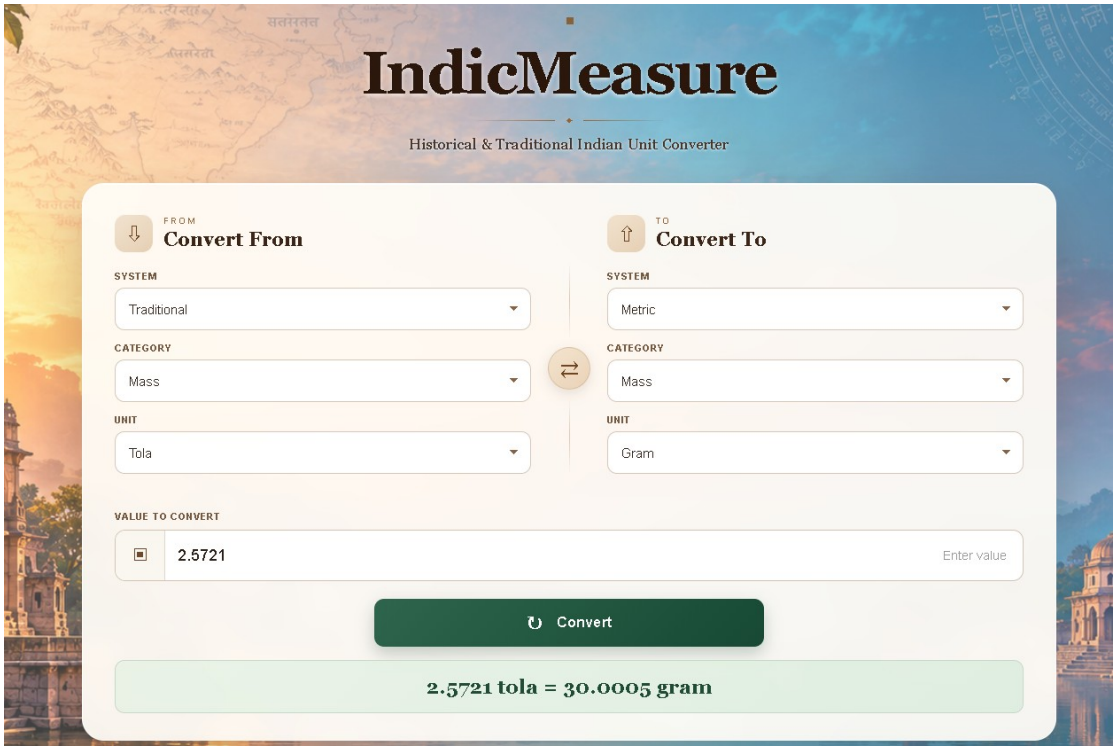
12. Images and Demonstration








IndicMeasure
Historical & Traditional Indian Unit Converter
Convert From
SYSTEM
Metric
CATEGORY
Mass
UNIT
Gram
Convert To
SYSTEM
Traditional
CATEGORY
Mass
UNIT
Tola
VALUE TO CONVERT
30
Enter value
Convert
30 gram = 2.5721 tola
Unit Information
Gram
System: Metric
Category: Mass
Description: Metric base unit of mass.
Historical / Regional Usage: International metric system
Conversion History
30 gram → 2.5721 tola
Clear History


IndicMeasure
Historical & Traditional Indian Unit Converter
FROM
Convert From
SYSTEM
Traditional
CATEGORY
Mass
UNIT
Tola
TO
Convert To
SYSTEM
Metric
CATEGORY
Mass
UNIT
Gram
VALUE TO CONVERT
2.5721
Enter value
Convert
2.5721 tola = 30.0005 gram

PROJECT REPOSITORY

Source code and project files are available at:

GitHub Repository – Java Micro Project

https://github.com/JshlatD/Java-Micro-Project